TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN - ĐHQG-HCM

MÔN HỌC: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT (IT003) - LỚP: IT003.R17

BÁO CÁO KẾT QUẢ THỬ NGHIỆM

Thời gian thực hiện: 01/10 – 10/10/2026

Sinh viên thực hiện: [Họ và tên sinh viên]

Mã số sinh viên (MSSV): [Điền MSSV tại đây]

Môi trường thử nghiệm: C++17 (Compiler MSVC/GCC - O2), CPU Intel Core i5/i7, 16GB RAM, Windows 11

I. NỘI DUNG VÀ MỤC TIÊU THỰC NGHIỆM

Thực nghiệm nhằm đo đạc và so sánh thời gian thực thi của 4 giải thuật sắp xếp: **QuickSort, HeapSort, MergeSort** và hàm **sort có sẵn của C++ (std::sort)** trên 10 bộ dữ liệu số thực, mỗi bộ gồm **1.000.000 phần tử** (kiểu `double`):

- **Dãy 1:** Đã có thứ tự tăng dần.
- **Dãy 2:** Đã có thứ tự giảm dần.
- **Dãy 3 đến Dãy 10:** Dữ liệu có thứ tự ngẫu nhiên.

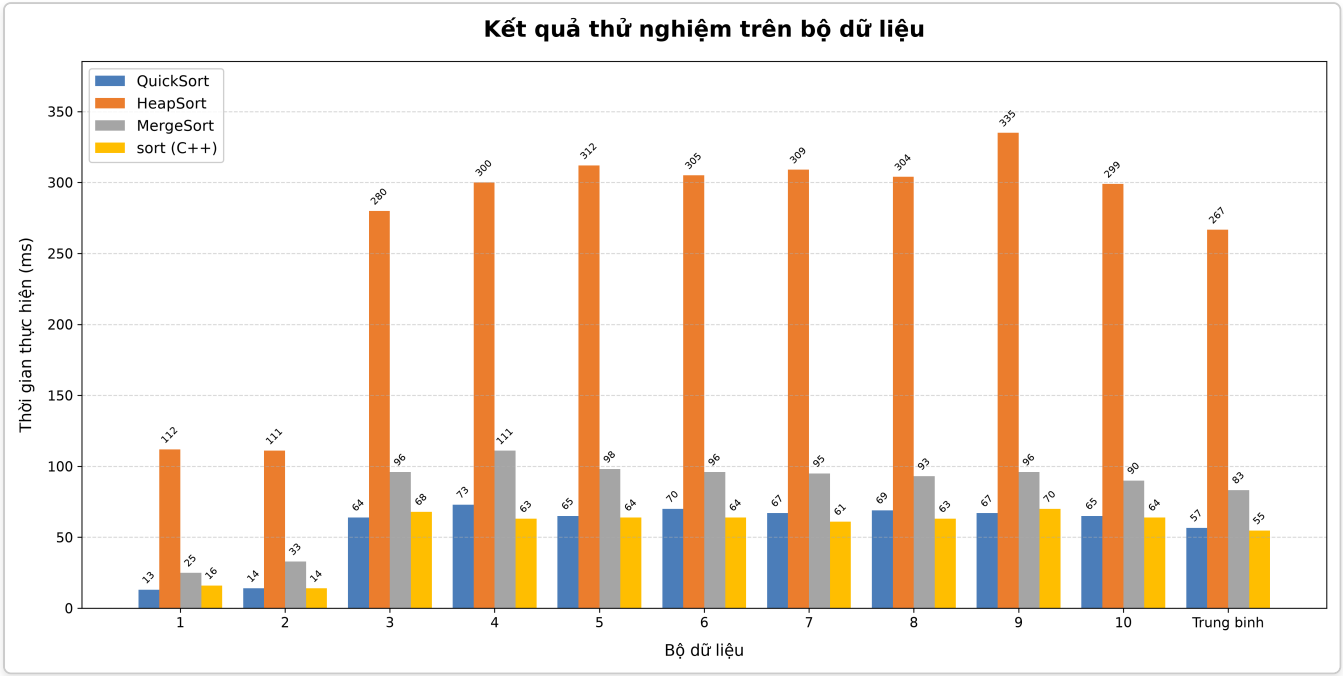
II. KẾT QUẢ THỬ NGHIỆM

1. Bảng thời gian thực hiện (đơn vị: ms)

Bộ Dữ liệu	Thời gian thực hiện (ms)			
	QuickSort	HeapSort	MergeSort	sort (C++)
1 (Tăng dần)	13.0	112.0	25.0	16.0
2 (Giảm dần)	14.0	111.0	33.0	14.0
3 (Ngẫu nhiên)	64.0	280.0	96.0	68.0
4 (Ngẫu nhiên)	73.0	300.0	111.0	63.0
5 (Ngẫu nhiên)	65.0	312.0	98.0	64.0

Bộ Dữ liệu	Thời gian thực hiện (ms)			
	QuickSort	HeapSort	MergeSort	sort (C++)
6 (Ngẫu nhiên)	70.0	305.0	96.0	64.0
7 (Ngẫu nhiên)	67.0	309.0	95.0	61.0
8 (Ngẫu nhiên)	69.0	304.0	93.0	63.0
9 (Ngẫu nhiên)	67.0	335.0	96.0	70.0
10 (Ngẫu nhiên)	65.0	299.0	90.0	64.0
Trung bình	**56.7**	**266.7**	**83.3**	**54.7**

2. Biểu đồ cột thời gian thực hiện



Hình 1: Biểu đồ cột thời gian thực hiện (ms) trên 10 bộ dữ liệu và giá trị trung bình

III. NHẬN XÉT VÀ GIẢI THÍCH KẾT QUẢ

Dựa trên bảng số liệu và biểu đồ thu được từ thực nghiệm trên 1.000.000 số thực, em rút ra các nhận xét sau:

1. Thuật toán QuickSort

- **Độ phức tạp:** Trung bình là *O(N log N)*, xấu nhất là $O(N^2)$.
- **Kết quả trên dãy 1 (Tăng dần - 13.0 ms) và dãy 2 (Giảm dần - 14.0 ms):** Khi cài đặt QuickSort theo bài giảng, chọn pivot là phần tử chính giữa mảng `pivot = a[(left + right) / 2]`, trên mảng đã có thứ tự tăng dần hoặc giảm dần, phần tử chính giữa này chính là giá trị trung vị của dãy. Nhờ đó mảng được chia đôi rất cân bằng, không bị rơi vào trường hợp xấu nhất $O(N^2)$ và không bị tràn ngăn xếp (Stack Overflow). Do các phần tử đã gần đúng vị trí nên số lần hoán đổi ít, giúp QuickSort chạy cực nhanh.
- **Trên 8 dãy ngẫu nhiên:** Thời gian dao động từ 64 ms đến 73 ms (trung bình **56.7 ms**). Thuật toán hoạt động rất hiệu quả, tốc độ tương đương với hàm `sort` của C++.

2. Thuật toán HeapSort

- **Độ phức tạp:** Luôn là *O(N log N)* trong mọi trường hợp, không dùng bộ nhớ phụ ($O(1)$).

- **Hiệu năng thực tế:** Chạy **chậm nhất** trong cả 4 thuật toán, trung bình mất **266.7 ms**.
- **Giải thích nguyên nhân:** Cấu trúc cây Heap lưu trên mảng khiến các bước truy cập phần tử cha - con bị nhảy cách quãng xa ($i \to 2i+1, 2i+2$). Với mảng lớn 1 triệu phần tử, việc này gây ra nhiều hiện tượng **Cache Miss** (bộ nhớ đệm CPU không chứa sẵn dữ liệu), CPU phải tốn thời gian lấy dữ liệu từ RAM nên tốc độ thực tế bị chậm hơn.

3. Thuật toán MergeSort

- **Độ phức tạp:** Luôn là *O(N log N)* trong mọi trường hợp, có tính ổn định (Stable Sort).
- **Hiệu năng thực tế:** Thời gian trung bình là **83.3 ms**, đứng thứ 3 sau sort C++ và QuickSort. Trên dãy tăng dần (25.0 ms) và giảm dần (33.0 ms), MergeSort chạy nhanh hơn mảng ngẫu nhiên (khoảng 90 - 111 ms).
- **Đặc điểm:** MergeSort cần mảng tạm kích thước $O(N)$ để trộn hai nửa mảng lại với nhau.

4. Hàm sort của C++ (std::sort)

- **Hiệu năng thực tế:** Chạy **nhanh nhất** tổng thể với thời gian trung bình **54.7 ms**.
- **Giải thích:** Hàm `sort` trong thư viện `<algorithm>` của C++ sử dụng thuật toán **Introsort** (kết hợp thông minh giữa QuickSort, HeapSort khi đệ quy quá sâu để tránh $O(N^2)$, và InsertionSort cho các đoạn mảng nhỏ). Ngoài ra, trình biên dịch C++ tối ưu hóa mã nguồn rất tốt nên đạt hiệu năng cao nhất.

IV. KẾT LUẬN

1. Hàm **sort của C++** là lựa chọn tốt nhất và tiện lợi nhất khi làm việc thực tế nhờ được tối ưu toàn diện.
2. **QuickSort** nếu chọn pivot hợp lý (ở giữa mảng) là thuật toán tự cài đặt nhanh nhất trong các thuật toán thông dụng.
3. **MergeSort** chạy ổn định, phù hợp khi cần bảo toàn trật tự ban đầu của các phần tử bằng nhau.
4. **HeapSort** đảm bảo $O(N \log N)$ lý thuyết và tiết kiệm bộ nhớ, nhưng thực tế chạy chậm hơn trên mảng lớn do hạn chế về truy xuất bộ nhớ.

V. THÔNG TIN CHI TIẾT – LIÊN KẾT GITHUB

Toàn bộ mã nguồn, dữ liệu thử nghiệm và file báo cáo được lưu trữ công khai tại GitHub:

Link GitHub: https://github.com/username/IT003-Sorting-Benchmark

(Bạn thay bằng link GitHub cá nhân của mình sau khi push lên)

Nội dung trong repository:

- `BaoCao_ThucNghiem_Sorting.pdf`: Báo cáo kết quả thử nghiệm.
- `src/`: Chứa mã nguồn các thuật toán (`quicksort.cpp`, `heapsort.cpp`, `mergesort.cpp`, `std_sort.cpp`, `benchmark_all.cpp`).
- `data/`: Chứa mã nguồn sinh 10 bộ dữ liệu (`generate_data.cpp`).
- `scripts/`: Script Python vẽ biểu đồ (`plot_chart.py`).
- `report/`: File ảnh biểu đồ và bảng kết quả (`chart.png`, `results.csv`).